

COMPANION TO THE BUILD PLAN

Remote Control Parts Catalog

Every open-source component the appliance needs, grouped by layer, with a verdict on each. Opinions included — a list of sixty repositories without them is just more work for you.

- USE

Build on this
- STUDY

Read it, don't depend on it
- CAREFUL

Real catch attached
- SKIP

Wrong tool here

THE SPINE

Nine repos carry the whole product

Everything else in this catalog is supporting cast. If you only cloned these nine, you would still have the shape of the machine.

Genymobile/scrcpy

Mirror and control the Android handset.
Solved problem — do not rebuild it.

mobile-dev-inc/maestro

Declarative YAML mobile flows. The shape
your flow library should take.

ggml-org/llama.cpp

Local inference, GGUF weights, LoRA
adapter loading.

mostlygeek/llama-swap

Hot-swaps models behind one OpenAI-
compatible endpoint. Your model manager,
already written.

rhasspy/wyoming

The protocol that wires wake word →
speech → intent → voice into one pipeline.

containers/bootc

The OS as a signed container image. Atomic
swap, automatic rollback.

sigstore/cosign

Sign every image and flow pack; publish to a public transparency log.

modelcontextprotocol

How agents call tools. Adopt the standard instead of inventing a plugin format.

home-assistant/operating-system

Your closest living analogue. Read this one before you write anything.

LAYER 00**Reference builds — read these first**

Four projects have already solved large parts of what you are attempting. Time spent reading them is the cheapest engineering you will ever do.

PROJECT	LICENSE	WHAT IT GIVES YOU	VERDICT
home-assistant/operating-system	Apache-2.0	The single most important repo in this catalog. An immutable appliance OS with A/B boot slots, RAUC over-the-air updates, and a non-technical owner on the other end. Same problem you have, already shipped to millions of devices.	STUDY
home-assistant/core	Apache-2.0	The automation engine, the device abstraction, and the Assist voice pipeline. Also the business template: Nabu Casa sells the hardware and a cloud subscription while the Open Home Foundation holds the project. That is your model with the serial numbers filed off.	STUDY
ubblue-os/bluefin · ubblue-os	Apache-2.0	Universal Blue builds polished, opinionated desktops as bootc images in public CI. The best worked example of the exact build pipeline you need for the OS layer.	USE

PROJECT	LICENSE	WHAT IT GIVES YOU	VERDICT
omacom/omarchy	MIT	Read manual/ for documentation standard, themes/ for how theming becomes a product feature, and install/hardware/ as a list of components to avoid. Do not adopt its mutable-Arch update model.	STUDY

LAYER 01

Immutable OS and updates

PROJECT	LICENSE	ROLE	VERDICT
containers/bootc	Apache-2.0	Your OS pipeline. Boot a machine directly from an OCI container image. Build in CI, sign, push; devices pull and swap atomically.	USE
ostreedev/ostree	LGPL-2.1	The content-addressed filesystem underneath bootc. Gives you file-level deduplicated deltas, so a full OS update downloads like a patch.	USE
fedora-iot/greenboot	LGPL-2.1	The auto-rollback. Runs health checks after boot; a failed check reverts to the previous image before the owner ever sees a problem. This is what makes updates unscary.	USE
rauc/rauc	LGPL-2.1	Classic A/B slot updater — what Home Assistant OS runs. The right choice only if you go Buildroot/Yocto embedded instead of OCI images.	CAREFUL
osbuild/osbuild	Apache-2.0	Turns your bootc image into installable media — ISO, raw disk, the factory flash artifact.	USE

PROJECT	LICENSE	ROLE	VERDICT
systemd-sysupdate	LGPL-2.1	Minimal A/B image updater already in systemd. Worth knowing, but bootc gives you more for the same effort.	STUDY
<u>mendersoftware/mender</u>	Apache-2.0	Full fleet OTA with a management server. Capable, but the server-side fleet model conflicts with your pull-only, no-access posture.	SKIP

LAYER 02

Boot integrity and signing

PROJECT	LICENSE	ROLE	VERDICT
systemd-boot + UKI	LGPL-2.1	Unified kernel images signed as one object. Simpler than GRUB2 and avoids its licensing entanglement entirely.	USE
<u>Foxboron/sbctl</u>	MIT	Secure Boot key management that a human can actually operate. Pair with <code>mokutil</code> so owners can enroll their own keys and unlock the machine.	USE
<u>sigstore/cosign</u>	Apache-2.0	Signs every OS image and flow pack. Devices verify before staging. Keys live in an HSM.	USE
<u>sigstore/rekor</u>	Apache-2.0	The transparency log. Devices refuse anything not publicly logged, which makes a targeted backdoor impossible rather than merely promised. This is the repo that turns your privacy claim into a proof.	USE
<u>project-zot/zot</u>	Apache-2.0	Lightweight OCI-native registry. Simpler to self-host than Harbor and enough for a fleet in the thousands.	USE

PROJECT	LICENSE	ROLE	VERDICT
fwupd/fwupd	LGPL-2.1	Firmware updates through LVFS. Cheap to add and it makes the appliance feel like a real product rather than a PC in a case.	USE
tpm2-software/tpm2-tss	BSD-2	Device identity sealed in the TPM. Backs entitlement without a user account, and disk encryption without a password prompt on a wall-mounted box.	USE

LAYER 03

Android remote control — your core IP

Everything else in this catalog is assembly. This layer is where your actual product gets built, so use these to learn the technique rather than to inherit someone's architecture.

PROJECT	LICENSE	ROLE	VERDICT
Genymobile/scrcpy	Apache-2.0	Mirroring and manual control. Low-latency Android screen on the Linux box with input passthrough. Ships a server-side component you can drive programmatically. Non-negotiable.	USE
AccessibilityService (Android SDK)		You write this yourself and it is the heart of the product. It hands you the semantic UI tree — real buttons with real IDs and text — plus gesture dispatch. Structure beats pixels by a wide margin.	USE
mobile-dev-inc/maestro	Apache-2.0	Read this for the flow format. Declarative YAML UI flows with built-in waiting and assertions — exactly the shape your flow library wants, and it already solved the tedious parts of element matching.	USE
openatx/uiautomator2	MIT	Python control of Android UI. The fastest path to a working prototype in week one, before you write your own service.	USE

PROJECT	LICENSE	ROLE	VERDICT
droidrun/mobilerun	open source	LLM-agnostic agent that inspects UI state, taps, types, and plans multi-step flows via CLI or Python. Closest working reference to your agent loop.	STUDY
sktyou/OpenGUI	open source	Android GUI agent built on standard AccessibilityService APIs, with a local REST API for remote driving. Good architecture reference for the service you will write.	STUDY
OpenBMB/AgentCPM-GUI	Apache-2.0	An 8B on-device GUI agent model that takes phone screenshots and executes tasks. Your vision fallback when the accessibility tree goes blind on a WebView or canvas.	USE
google-research/android-world	Apache-2.0	Your test harness. A benchmark of real Android tasks. Do not guess at your flow reliability — measure it against a standard suite and watch the number move.	USE
appium/appium	Apache-2.0	The mature, heavyweight option. Built for CI test farms, not a live appliance — the ceremony costs more than it returns here.	SKIP

PROJECT	LICENSE	ROLE	VERDICT
waydroid/waydroid	GPL-3.0	Android in a container on the Linux box. Tempting, but it forfeits the real SIM and gets flagged by exactly the POS and banking apps you need. The physical handset is the whole point.	SKIP

Test detection before you build anything on this layer

Some apps refuse to run with an active accessibility service. Install every app in your target vertical on a real handset, turn your service on, and see what still works. One afternoon, one phone, and it tells you what your addressable market actually is.

Local inference and the model catalog

PROJECT	LICENSE	ROLE	VERDICT
ggml-org/llama.cpp	MIT	The engine. GGUF weights, quantization tooling, LoRA adapter loading, an OpenAI-compatible server, and first-class AMD support through ROCm and Vulkan.	USE
mostlygeek/llama-swap	MIT	Your model manager, already written. A single Go binary that sits in front of llama.cpp or vLLM, inspects the requested model, starts the right process, proxies the call, and evicts idle instances. Process isolation means one model crashing cannot take the others down.	USE
vllm-project/vllm	Apache-2.0	Higher throughput and multi-LoRA serving, at the cost of a much heavier runtime. Right for a business box serving several staff at once; overkill for a single owner.	CAREFUL
ollama/ollama	MIT	Easiest developer experience and great for your first two weeks. Hands you less control over quantization and memory than you will eventually want.	CAREFUL

PROJECT	LICENSE	ROLE	VERDICT
huggingface/huggingface_hub	Apache-2.0	The client behind your one-click catalog. You mirror, quantize, test on your own SKU, sign, and serve — the customer never sees a model card or a config file.	USE
ROCm/ROCm	MIT	The AMD compute stack for Strix Halo. In-tree kernel driver is precisely what keeps your single-image OS architecture clean — the reason to pick AMD over NVIDIA here is architectural, not benchmark-driven.	USE
open-webui/open-webui	BSD-3	A complete chat front end you can point at your stack today. Useful as internal tooling and as a design reference; not the ten-foot UI you will ship.	STUDY

LAYER 05

Voice — speak to it and it works

Do not assemble this from scratch. The Wyoming protocol already defines the whole pipeline — wake word, speech-to-text, intent, speech-out — as swappable services, and it has been running in millions of homes for years.

PROJECT	LICENSE	ROLE	VERDICT
rhasspy/wyoming	MIT	The glue. A simple protocol connecting each voice stage as an independent service. Adopt it and every component below becomes swappable without rewriting your pipeline.	USE
dscripka/openWakeWord	Apache-2.0	Wake word detection. Train your own brand word — the phrase people say to your box is part of the product, not a config value.	USE
snakers4/silero-vad	MIT	Voice activity detection — knowing when the speaker stopped. Small, fast, and the difference between a natural exchange and an awkward one.	USE
SYSTRAN/faster-whisper	MIT	Speech to text. Sub-second on real hardware. Pair with whisper.cpp if you want one inference runtime across the whole box.	USE
hexgrad/kokoro	Apache-2.0	Text to speech, 82M parameters. Natural output at a size that runs alongside everything else. This is your voice.	USE

PROJECT	LICENSE	ROLE	VERDICT
rhasspy/piper	MIT	Archived October 2025 and read-only. Still works, still recommended in old tutorials — do not start a new product on it. Go to Kokoro.	SKIP
pipecat-ai/pipecat	BSD-2	Real-time conversational voice with interruption handling and barge-in. Reach for it when the wall box needs to hold a phone conversation rather than answer a command.	STUDY
speaches-ai/speaches	MIT	Wraps local speech-to-text and text-to-speech behind an OpenAI-compatible API. Convenient seam if you ever want to swap a cloud voice in for a customer who asks.	USE

LAYER 06

Orchestration, tools and automations

PROJECT	LICENSE	ROLE	VERDICT
modelcontextprotocol	MIT	How agents call tools. Spec plus SDKs plus a library of existing servers. Every capability on your box becomes an MCP server, and you inherit an ecosystem instead of inventing a plugin format.	USE
node-red/node-red	Apache-2.0	Visual flow automation with a huge node library, genuinely open licensed, and light enough to run on the appliance. The right engine under your trigger-and-action layer.	USE
n8n-io/n8n	Sustainable Use	Excellent product, but not open source — its license restricts commercial hosting and redistribution. Shipping it inside a product you sell needs a commercial agreement. Know this before you build on it.	CAREFUL

PROJECT	LICENSE	ROLE	VERDICT
temporalio/temporal	MIT	Durable execution — a flow survives a crash, a reboot, or a phone that fell asleep mid-task, and resumes exactly where it stopped. Heavy, but retries and resumability are not optional when real invoices are involved.	CAREFUL
langchain-ai/langgraph	MIT	Stateful agent graphs with checkpointing and human-in-the-loop interrupts — which maps almost directly onto your approval board.	STUDY

LAYER 07

Interface, apps and data

PROJECT	LICENSE	ROLE	VERDICT
<u>flutter/flutter</u>	BSD-3	One codebase for the wall screen and the phone app. Permissive license with no commercial-embedded complications, unlike the Qt/QML route Omarchy took — which matters the moment you sell hardware.	USE
<u>cage-kiosk/cage</u>	MIT	A Wayland compositor that runs exactly one application fullscreen. Your ten-foot UI in kiosk mode, with no desktop for anyone to fall out of.	USE
<u>flatpak/flatpak</u>	LGPL-2.1	Sandboxed apps versioned separately from the OS. Pair with <u>xdg-desktop-portal</u> so file, camera and mic access is granted per app by a person, not assumed.	USE
<u>89luca89/distrobox</u>	GPL-3.0	The escape hatch. Full mutable Linux with root in a container, quarantined from the host. This is how you offer total freedom without any risk to the appliance.	USE

PROJECT	LICENSE	ROLE	VERDICT
asg017/sqlite-vec	Apache-2.0 / MIT	Vector search inside SQLite. One file holds the business's whole memory — easy to encrypt, easy to back up, easy to hand back when they leave.	USE
restic/restic	BSD-2	Encrypted backup to a destination the customer chooses — their NAS, their bucket. They are protected from a dead drive; you never hold a key.	USE
juanfont/headscale	BSD-3	Self-hosted control plane for a WireGuard mesh, so the phone app reaches the box directly. Keeps your "no pipe into the customer's machine" posture intact.	USE

ASSEMBLY

What to clone this week

In order. Each step produces something you can see working before the next one starts.

-
- 01** **scrcpy** — get a real phone mirrored on a Linux box with working input. One afternoon.
 - 02** **uiautomator2** — dump the UI tree of the Google Business Profile app and tap a button from Python. Prototype before you write a service.
 - 03** **maestro** — read the YAML flow format and steal its element-matching and waiting logic. Don't solve that twice.
-

- 04 **llama.cpp + llama-swap** — one endpoint, two models, hot swap on demand. Your orchestrator's engine room, running in an hour.

- 05 **wyoming + openWakeWord + faster-whisper + kokoro** — wire the four together and say a sentence to your box. This is the demo people remember.

- 06 **home-assistant/operating-system** — read the update system docs end to end before you commit to your own OS architecture.

- 07 **bootc + greenboot** — build one image, boot it, break it on purpose, watch it roll itself back.

The order matters more than the list

Steps one through three are the product. Steps four and five are the demo. Steps six and seven are the company. Anyone can clone all sixty repos in this catalog in a weekend; the useful question is which three you have working by Friday.

Licenses noted where they affect shipping hardware commercially — verify each against its repository before you depend on it, since projects relicense. Two flagged above are live traps: Piper is archived, and n8n is not open source.